

B.Sc. Computer Science · Year 1, Semester 1
SCS1303

Introduction to Software Engineering

Section 2 · Software Processes

Prof. K. P. Hewagamage

University of Colombo School of Computing

Section 2 — Intended Learning Outcomes

By the end of this section, students should be able to:

- LO1 Explain what a software process is and why a defined process matters
- LO2 Distinguish between plan-driven and agile development cultures
- LO3 Describe the Waterfall model — phases, strengths, and limitations
- LO4 Explain Incremental, Iterative, V&V, Prototyping, Spiral, RAD, and Reuse-Oriented models
- LO5 Explain the Unified Process as a hybrid model bridging both cultures
- LO6 Identify the four core process activities present in every software process
- LO7 Describe the DevOps culture and its role in modern software delivery
- LO8 Recognise how AI is beginning to influence software development processes

Section 2 — Roadmap

2.3 is the core of this section — covered in detail for each model:

2.3 covers: Waterfall · Incremental · Verification & Validation · Prototyping · Spiral · RAD · Reuse-Oriented

2.1

What is a software process?

2.2

Two cultures: Plan-driven vs Agile

2.3

Plan-Driven Process Models

2.4

The Unified Process

2.5

Agile culture — introduction

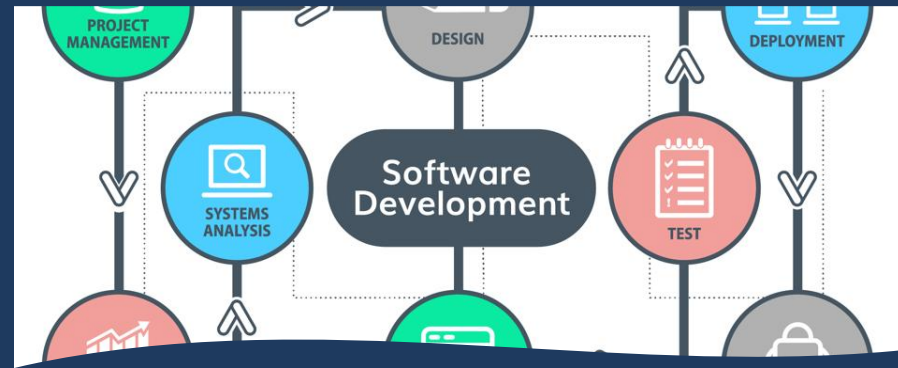
2.6

DevOps and AI in software processes

2.1

What is a Software Process?

The foundation before choosing any model



Is there one Process , One Road Map, One predefine template that will take through all the necessary steps from start to finish to make a piece of software?



What is a Software Process?

A structured set of activities required to develop a software system.

It defines what to do, in what order, who is responsible, and how to verify it has been done correctly.

1

Activities

The work that is carried out — specifying, designing, coding, testing, deploying, maintaining

2

Products

What each activity produces — requirement documents, design models, tested source code, manuals

3

Roles

Who carries out the work — analyst, architect, developer, tester, project manager, client

4

Pre- and Post-conditions

What must be true before an activity starts (pre), and what it must produce or guarantee (post)

Why Do We Need a Defined Process?

Without a shared process, no common understanding, no measurement, no improvement

- | | | |
|---|--------------------------------|--|
| 1 | Common understanding | Writing down a process creates a shared language for the team. Everyone understands the activities, responsibilities, and constraints — no hidden assumptions, no working at cross-purposes. |
| 2 | Consistency and quality | A defined process reduces defects and makes quality repeatable. Good results should not depend on which developer happens to be working on a feature today. |
| 3 | Measurable progress | Without a process, you cannot tell if a project is on track. A defined process provides milestones, deliverables, and checkpoints that make progress visible to everyone. |
| 4 | Adaptability | Process models help a team understand that different systems need different approaches. No single model fits every situation — but having models lets you choose wisely. |
| 5 | Professional practice | Software engineering is a discipline, not a craft. A defined process is what separates professional engineering from ad hoc programming. |

2.2

Two Cultures of Software Development

Plan-driven vs Agile — the fundamental divide

1. Software process models
2. Process activities
3. Coping with change
4. Process Improvement

Two Cultures of Software Development

All process models belong to one culture — or deliberately blend both

"Culture" means the values, assumptions, and shared ways of working that shape how a team approaches building software.

Plan-Driven Culture

- Plan everything upfront before any coding begins
- Requirements, design, schedule fixed early
- Progress measured strictly against the plan
- Change is controlled and expensive
- Emphasis on documentation and formal reviews
- Best when requirements are stable and well understood

vs

Agile Culture

- Embrace change — requirements will evolve
- Planning is incremental and continuous
- Working software delivered frequently in short cycles
- Customer collaboration over contract negotiation
- Self-organising, cross-functional small teams
- Best when requirements are uncertain or evolving

Software Process Models — Mapped to Their Culture

Plan-Driven Models

- Waterfall
- Incremental Development
- Verification & Validation
- Prototyping (Throwaway)
- Spiral Model
- RAD Model
- Reuse-Oriented (CBSE)

Hybrid

Unified Process (UP)

Bridges both cultures deliberately

Agile Models

- Scrum
- Extreme Programming (XP)
- Kanban
- Lean Software Development
- Feature-Driven Dev (FDD)
- SAFe (Scaled Agile)
- DSDM

In practice, most large systems use a process that blends elements from both cultures. Section 2.5 introduces Agile. Section 3 covers it in full depth.

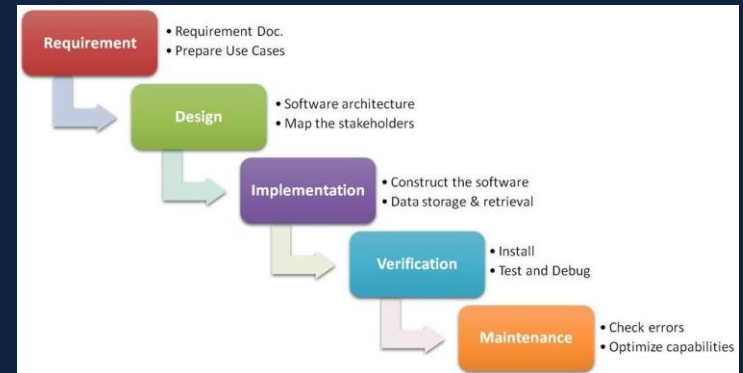
2.3

Plan-Driven Process Models

Seven models — each solving a different problem

The Waterfall Model

Sequential · Document-driven · One phase at a time



The Waterfall Model — Overview

The first formally published software development model (Royce, 1970)

The Waterfall model divides the development process into a set of distinct phases that flow downward like a waterfall. Each phase must be completed and its output formally approved before the next phase begins.

1. Requirements Analysis & Definition

2. System & Software Design

3. Implementation & Unit Testing

4. Integration & System Testing

5. Operation & Maintenance

Note: Royce himself observed in 1970 that pure sequential development is 'risky and invites failure' — the model was a starting point, not the final answer.

Waterfall — Phase 1: Requirements Analysis and Definition

Phase 1 of 5 · Output: System Specification Document

The system's services, constraints, and goals are established by consultation with system users. These must be defined in detail and serve as a complete system specification.

What happens in this phase

- Identify all system stakeholders — users, customers, operators, regulators
- Gather requirements through interviews, workshops, observation, and review of existing systems
- Define functional requirements: what the system must do
- Define non-functional requirements: performance, security, reliability, usability
- Document all requirements in a formal specification
- Gain formal sign-off before proceeding

Key deliverables

- System Requirements Specification (SRS) document
- Functional requirements list
- Non-functional requirements list
- Feasibility study (technical and financial viability)
- Initial risk identification
- Stakeholder sign-off document

Key principle: The more precisely requirements are captured here, the less expensive change is later. Every ambiguity in this phase multiplies downstream.

Waterfall — Phase 2: System and Software Design

Phase 2 of 5 · Output: Design Specification Document

The requirements are allocated to either hardware or software subsystems. An overall system architecture is established, and the software design identifies and describes the fundamental software abstractions and their relationships.

Architectural design

Identify the overall structure — the principal components (subsystems/modules), their relationships, and how they are distributed across hardware.

Interface design

Define the interfaces between system components — how modules communicate, data formats exchanged, API contracts.

Component design

Each system component's internal design — algorithms, data structures, and logic for each module.

Database design

Design the data structures and their representation in a database — schema, tables, relationships, constraints.

The design document becomes the blueprint for the next phase. Programmers should be able to code directly from it without revisiting the requirements document.

Waterfall — Phase 3: Implementation and Unit Testing

Phase 3 of 5 · Output: Verified software units / modules

During this stage, the software design is realised as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.

What programmers do

- Translate each design component into source code
- Follow coding standards agreed by the team
- Test each unit (function, class, module) in isolation
- Fix defects found during unit testing
- Record and document all defects and resolutions
- Produce unit test reports for review

Unit testing — what it means

Each program unit (function, class, module) is tested independently against the specification for that unit. Errors found here are cheap to fix — no integration has happened yet.

Key deliverables

- Completed, compilable source code
- Unit test cases and results
- Code review records

Waterfall — Phase 4: Integration and System Testing

Phase 4 of 5 · Output: Tested, accepted software system

Individual program units are integrated and tested as a complete system to ensure that software requirements have been met. After testing, the system is delivered.

Integration testing

Individual modules are combined and tested together. Checks that interfaces between components work correctly and that data flows properly between them.

System testing

The complete, integrated system is tested against the original requirements specification. Checks that the system behaves correctly as a whole — including performance, security, and usability.

Acceptance testing

The customer tests the system with real-world data to verify it meets their needs. A formal sign-off is obtained from the customer. This is the last gate before delivery.

Regression testing

Any time a defect is fixed and the system is retested, regression testing checks that the fix did not break anything that was working before.

Testing is the most commonly used V&V activity. In Waterfall, it happens late — which means defects found here are expensive to fix. This is one of Waterfall's biggest weaknesses.

Waterfall — Phase 5: Operation and Maintenance

Phase 5 of 5 · Normally the longest lifecycle phase

The system is installed and put into practical use. Maintenance involves correcting errors not discovered earlier, improving system implementation, and enhancing services as new requirements emerge.

Corrective maintenance

20%

Fix bugs found by real users in the live system — defects that escaped testing

Adaptive maintenance

25%

Adapt the system to new hardware, OS updates, changed regulations, or APIs

Perfective maintenance

50%

Improve performance, add new features, improve usability based on user feedback

Preventive maintenance

5%

Improve reliability and maintainability before failures occur — refactoring

Maintenance typically consumes 60–80% of total software lifecycle cost. Software that is well-designed and documented costs significantly less to maintain.

Waterfall — Strengths and When to Use It

Strengths

- Simple to understand and manage — phases and outputs are clear
- Heavy documentation makes the project easy to hand over
- Easy to measure progress — each phase has a defined output
- Works well with fixed-price, fixed-scope contracts
- Suitable for regulated industries requiring formal sign-off

When to use Waterfall

- Requirements are clearly understood and unlikely to change
- Short projects with small, co-located teams
- Safety-critical systems — aviation, medical devices, defence
- Technology and tools are well-understood and stable
- Replacing a known, well-documented existing system

Real example: NASA space shuttle software (4 million lines of code, 0.1 defects per 1,000 lines) used a strict Waterfall process with formal reviews at every phase. The approach was justified by the extreme safety requirements and stable technical environment.

Waterfall — Limitations and Problems

Requirements never really stand still

In practice, stakeholders change their minds, business needs shift, and the world moves on. The Waterfall assumption that requirements can be fully defined upfront is rarely true for complex systems.

Real projects don't follow the sequence

Developers always discover problems in earlier phases as work progresses. In Waterfall, going back means expensive re-entry, re-approval, and rework of all subsequent phases.

Customer sees nothing until very late

The first working version of the system is not visible until Integration and System Testing — after all development is complete. If the customer's needs were misunderstood, discovering it here is catastrophic.

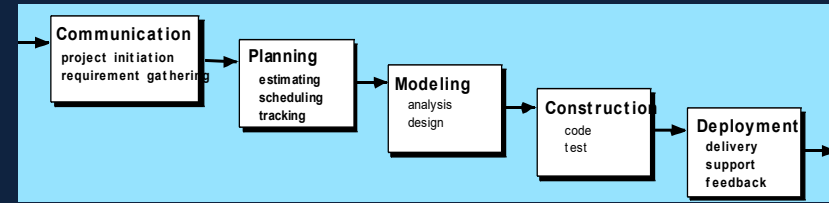
Testing happens at the end

Defects found late in the cycle are far more expensive to fix than those found early. A bug in requirements, only caught during acceptance testing, may require redesigning large parts of the system.

Assumes natural uncertainty can be eliminated

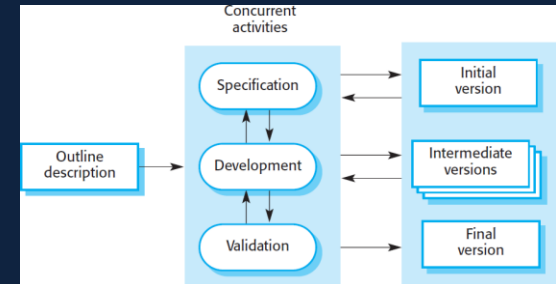
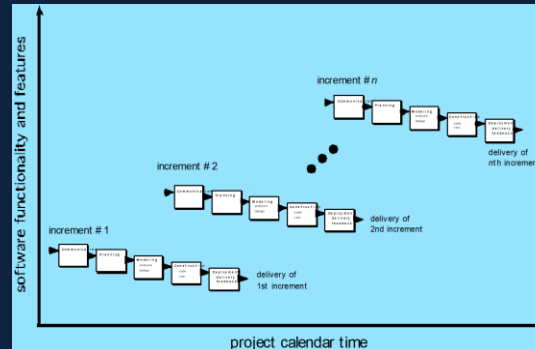
At the start of any project there is natural uncertainty. Waterfall requires treating this uncertainty as if it does not exist — forcing artificial precision that often proves wrong.

"The fundamental problem of the waterfall model is its emphasis on complete understanding of the problem before solving it — something that is rarely achievable." — Sommerville



Incremental Development

Build piece by piece · Deliver working software early · Reduce risk

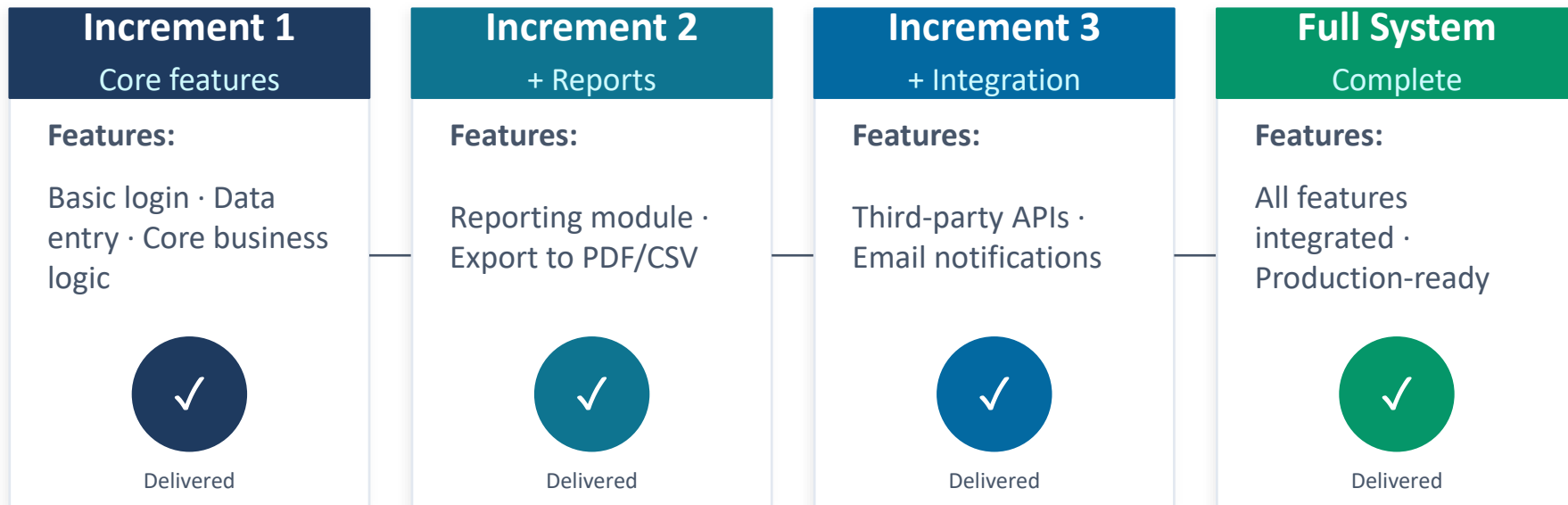


Incremental Development — Concept

Break the system into increments — each adds real, usable functionality

Incremental development is based on developing an initial implementation, getting feedback from users, and evolving the software through multiple versions until the required system is complete.

Highest-priority requirements are addressed in early increments. Customers can use and react to each increment.



Incremental vs Iterative — An Important Distinction

These terms are often confused — they mean different things

Incremental development

A scheduling and staging strategy:

The system is divided into parts that are developed at different times or rates, and integrated as they are completed.

Each increment adds new functionality — the output is typically released to users.

Key question: "What new feature does this increment add?"

The alternative is 'big bang' integration — build everything, then integrate all at once. This is high-risk.

Iterative development

A rework scheduling strategy:

Time is set aside to revisit and improve parts of the system that have already been built.

The output of an iteration is usually examined internally for modification, not necessarily released to users.

Key question: "How can we improve what we have already built?"

Agile combines both: iterations improve quality; increments add features.

Incremental Delivery

Deploy each increment to real users — gather feedback, refine the next increment

Incremental delivery goes one step further than incremental development — each completed increment is actually deployed into the customer's working environment, not just delivered internally.

1

Define services and priorities

Customers identify which services are most important. These drive the ordering of increments.

2

Define increment 1 requirements

Requirements for the first increment are defined in detail. Later requirements may still be vague.

3

Develop and deploy increment 1

Increment 1 is built, tested, and deployed into the real working environment.

4

Users experiment and give feedback

Real users work with the system. Their experience clarifies requirements for later increments.

5

Define next increment

Requirements for increment 2 are now refined, informed by actual user experience.

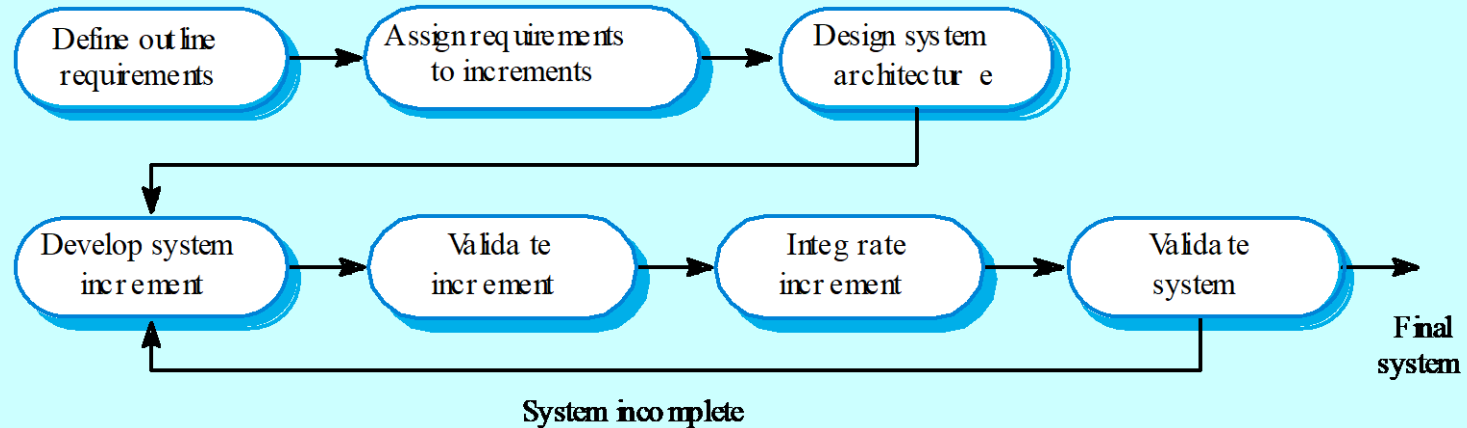
6

Repeat until complete

Continue until the system meets all requirements. Each cycle improves the result.

Important constraint: Once an increment is under development, requirements for that increment are frozen. Changes go into future increments.

Incremental development model



Incremental Development — Benefits and Problems

Benefits

- Working software delivered early — customers get value before the full system is complete
- Early increments act as prototypes — help clarify requirements for later increments
- Lower risk of total project failure — problems in one increment don't kill the whole project
- Highest-priority features receive the most testing — they are in the system longest
- Easier to manage change — new requirements go into future increments

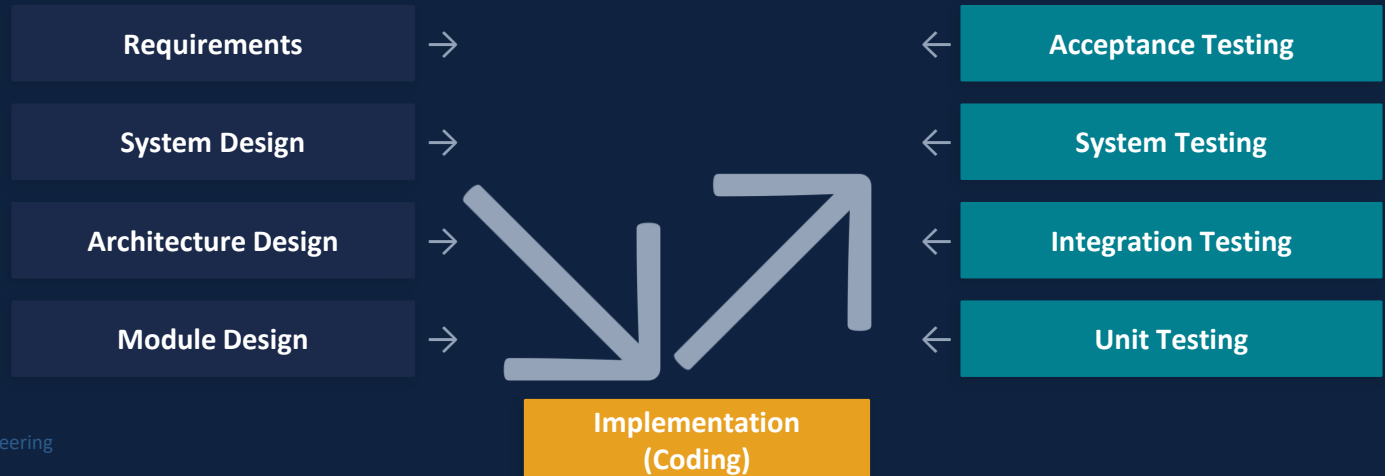
Problems

- Process is less visible — managers need documents to measure progress; frequent small releases produce fewer formal records
- System structure tends to degrade as new increments are added — unless time is spent on refactoring
- Hard to replace an existing system — users need all the old functionality before they will switch
- Difficult to identify common infrastructure needed by all increments when requirements are not yet defined

Incremental development is at the heart of most modern approaches —
Agile, Scrum, and DevOps all build on this fundamental idea.

Verification and Validation

Checking the system is built right · and the right system was built



Verification and Validation (V&V)

Two different but complementary questions about software quality

Verification

"Are we building the product right?"

Verification checks that the software conforms to its specification. We are checking internal consistency — does what we built match what was specified?

Methods:

- Code reviews and walkthroughs
- Formal inspections
- Static analysis
- Checking against design documents

Verification can be done without running the software — it is about logical correctness relative to the specification.

Validation

"Are we building the right product?"

Validation checks that the software meets the real needs of its users. Even if the system perfectly matches the specification, the specification itself may be wrong.

Methods:

- Testing with real users
- Acceptance testing with customer data
- User feedback and evaluation
- Prototype evaluation

Validation can only be fully done with real users — because only they know what they really need.

Classic quote: "Verification is checking the system against a document. Validation is checking the document against reality."

V&V — Stages of Testing

Testing is the most widely used V&V activity — it occurs at every stage of development

Each testing stage catches different types of defects. In Waterfall, these stages happen sequentially near the end. In Agile, they are woven throughout development.

1

Unit / Component Testing

Who: Developers

Individual functions, classes, and modules are tested in isolation. Confirms each unit behaves exactly as its specification requires.

Catches: Logic errors, off-by-one bugs, null references, boundary violations

2

Integration Testing

Who: Developers / QA team

Modules are combined progressively and tested together. Detects interface mismatches — where one module's output does not match another's expected input.

Catches: Interface bugs, data format mismatches, incorrect API calls, timing issues

3

System Testing

Who: QA team

The complete integrated system is tested against all functional and non-functional requirements. First time the entire system runs together.

Catches: Missing features, performance failures, security weaknesses, usability problems

4

Acceptance Testing

Who: Customer / Users

Customer tests the system with real data in conditions representative of real use. The formal gate before delivery. The customer must sign off.

Catches: Requirements misunderstandings, missing business rules, usability gaps

V&V — Inspections and Reviews

Non-testing V&V techniques that catch defects before execution

Testing is not the only way to find defects. Software inspections and reviews check documents and code without executing the software — and are often more cost-effective than testing alone.

Code review

Team members read and examine each other's code systematically. The most commonly used inspection technique. Catches logic errors, style violations, security issues, and misunderstandings early.

Formal inspection

A structured meeting where a team examines a document (requirements, design, or code) against a checklist. A recorder notes every defect. Studies show inspections catch 60–90% of defects before testing.

Walkthrough

The author presents their work to a group who ask questions and identify issues. Less formal than inspection. Good for early-stage documents where formal checklist is not yet appropriate.

Static analysis

Automated tools examine source code without running it. They check for known bug patterns, security vulnerabilities, and coding standard violations. SonarQube, FindBugs, ESLint are common examples.

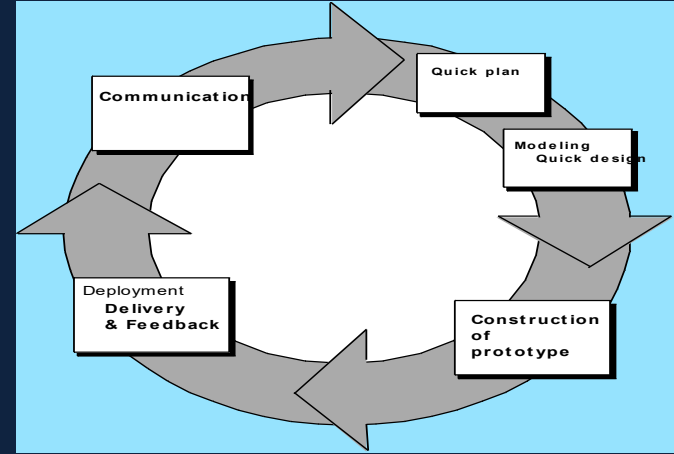
Fagan (1976) showed that formal code inspections reduced IBM's testing costs by 25% and delivered 40% fewer defects to system testing.

Plan-Driven Model · 2.3.4

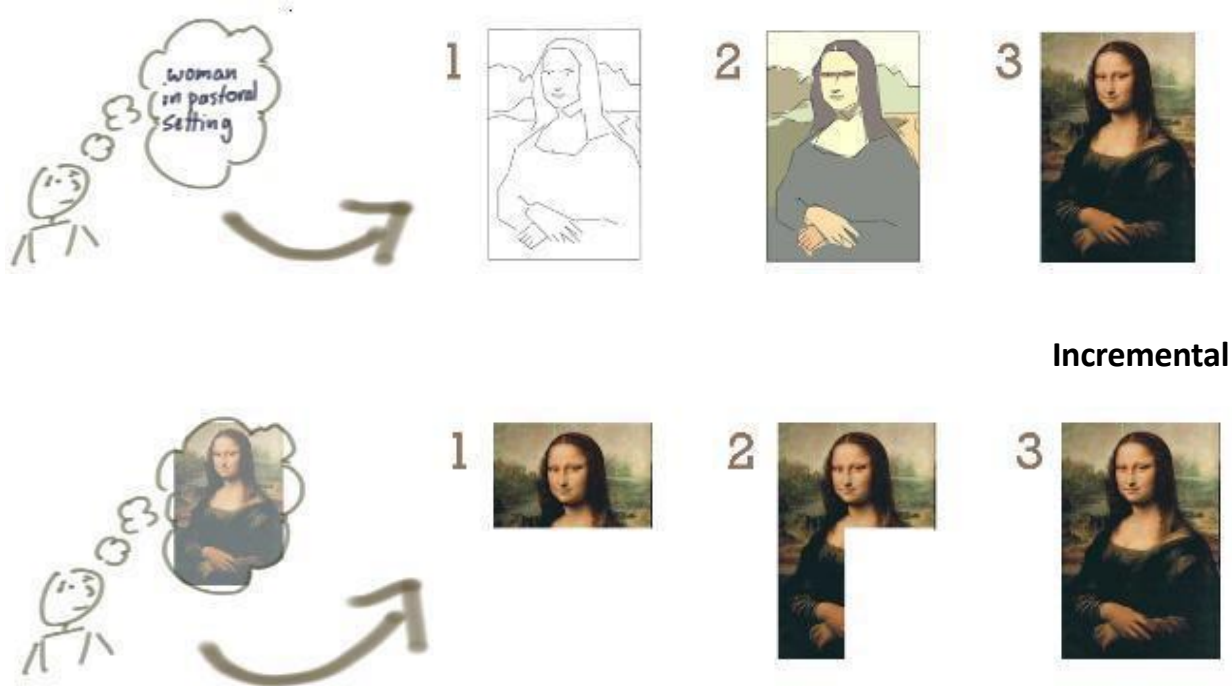
Prototyping

An Evolutionary Model

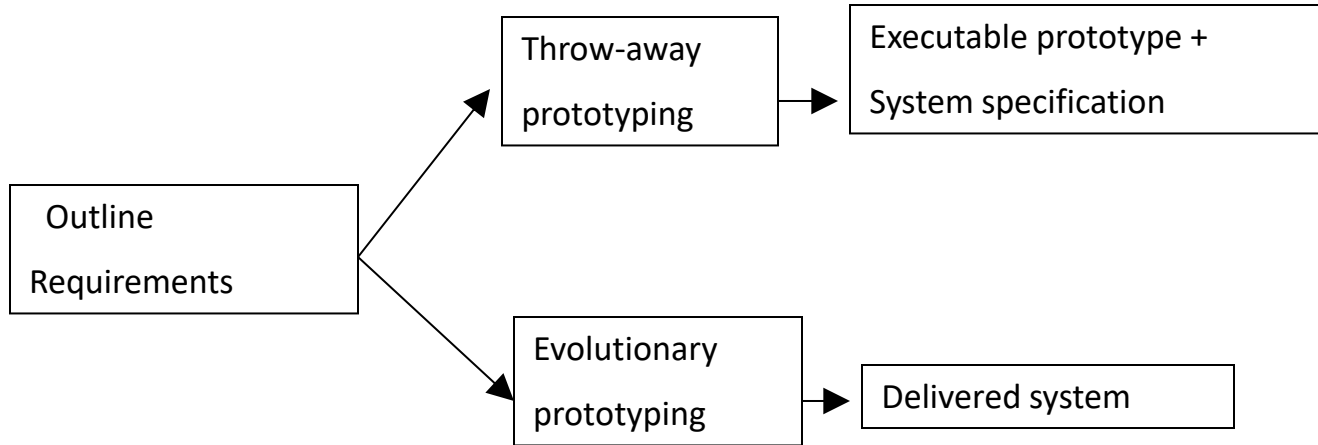
Build a quick version first · Clarify requirements · Reduce risk before full commitment



Incremental vs Iterative (Evolutionary)



Throw-away and Evolutionary Prototyping



Prototyping — Overview and Purpose

A prototype is an initial version of a system built to demonstrate concepts and try out design options

It is very difficult for end-users to anticipate how they will use new software systems to support their work. If the system is large and complex, it is probably impossible to make this assessment before the system is built. A prototype can be used to clarify vague requirements and evaluate design options with real users.

Three contexts where prototyping is most valuable:

Requirements engineering

Prototype helps with requirements elicitation and validation. Users react to what they see — revealing requirements they could not articulate in a meeting.

System design exploration

Prototype different architectural or UI options to evaluate trade-offs before committing to a design. Especially useful for user interface design.

Feasibility demonstration

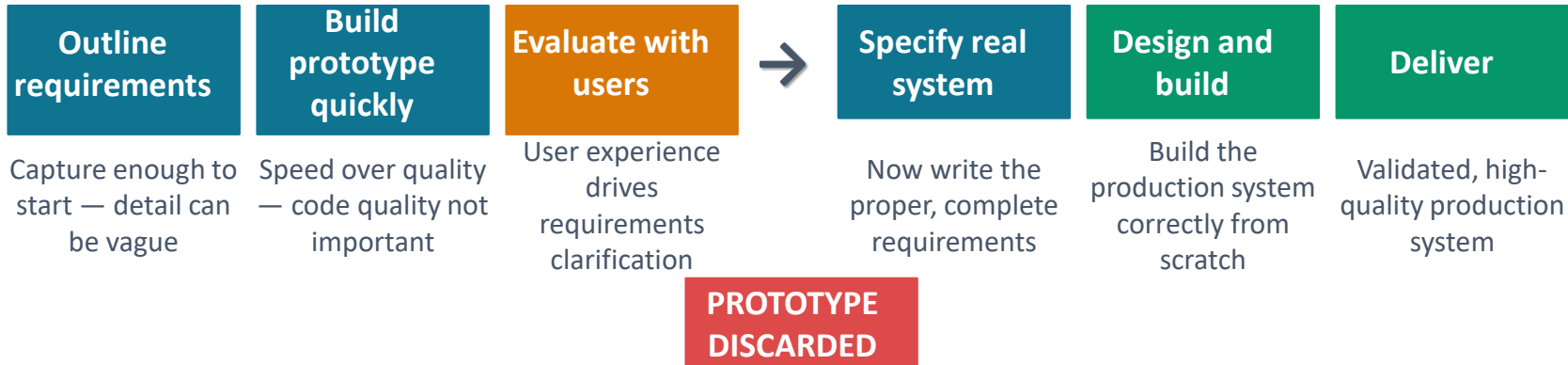
A proof-of-concept prototype demonstrates that a technical approach is achievable — before the full project investment is approved.

A prototype should always be evaluated with user participation. Building a prototype without user involvement defeats its purpose.

Throwaway Prototyping

Build quickly to learn — then discard and build properly

A throwaway prototype is built quickly with the goal of clarifying ambiguous requirements. It is evaluated by the customer and then discarded. The real system is then built from scratch using the knowledge gained.



Why discard it?

A prototype is built for speed — not quality. It will have no error handling, no security, no documentation, no maintainability. Deploying it to production would create long-term technical debt. It has served its purpose: to teach you what to build.

Benefits of throwaway prototyping

- Reduces requirements risk — mistakes caught early
- Users can react to something visible — not abstract
- Better UI/UX design before expensive development

Evolutionary Prototyping

Grow the prototype into the final system through repeated user feedback cycles

In evolutionary prototyping, the prototype is NOT discarded. Instead, it is continuously refined and evolved based on customer feedback until it becomes the delivered product.

Cycle 1

Build abstract specification and first prototype



Cycle 2

Evaluate prototype with users — gather feedback



Cycle 3

Refine prototype based on feedback



Cycle N

System adequate?
Yes → Deliver ·
No → Loop

↑ Loop if not adequate

Best suited for:

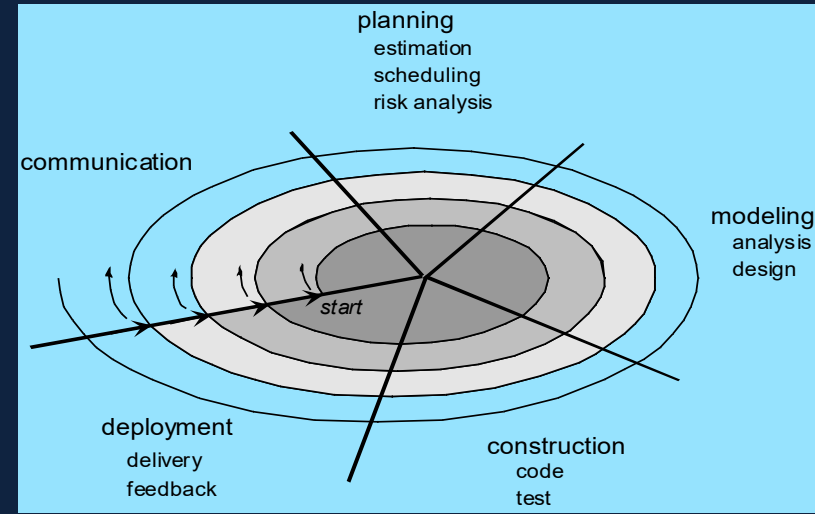
- Systems with rapidly changing or unclear requirements
- Projects where early delivery of core functionality has high value
- Smaller systems where full re-engineering each cycle is feasible

Risks:

- System structure may degrade without deliberate architectural care
- Contractual difficulties — hard to specify price and schedule upfront
- Prototype speed may create pressure to skip proper documentation

Plan-Driven Model · 2.3.5

The Spiral Model



Risk-driven · Each loop is a phase · Combines planning with prototyping



Incremental: Build one part completely at a time based on the original idea.



Iterative: Build a rough version, validate this and continue building bilding more quality and functionality in.



The Spiral Model — Overview

Boehm (1988) — designed specifically for large, complex, high-risk projects

The Spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the Waterfall model. Software is developed in a series of incremental releases — early iterations may be paper models or prototypes.

The most important feature of the Spiral model: risk assessment is built into every loop.

Quadrant 1

Determine objectives, alternatives, constraints

Define the objectives for this loop. Identify alternative approaches. Identify constraints — technical, cost, schedule, interfaces.

Quadrant 2

Evaluate alternatives, identify and resolve risks

Critically analyse each alternative for risk. Develop prototypes to address the highest-risk elements. Risk resolution drives what happens in this quadrant.

Quadrant 3

Develop and verify the next-level product

Build and test the product for this loop. May involve the full Waterfall sequence — requirements, design, code, test — for this increment.

Quadrant 4

Plan the next iteration

Review progress with the customer. Make the go/no-go decision for the next loop. Plan the next iteration — define objectives, alternatives, constraints.

Spiral Model — Risk as the Central Driver

Every loop begins and ends with risk assessment — unlike any other plan-driven model

In every other plan-driven model, risk is acknowledged but not systematically managed. The Spiral model makes risk management the central activity of every phase.

The deliverables of each loop, especially in early loops, must be selected specifically to address the greatest risks first.

Technical risk	Can we build this at all? Does the technology exist?	Proof-of-concept prototype in loop 1
Requirements risk	Do we understand what the customer really needs?	Throwaway prototype evaluated with users
Schedule risk	Can we deliver in the time available?	Realistic planning and milestone tracking
Budget risk	Will the cost exceed available funding?	Early cost estimation with regular review
Integration risk	Will the components work together as a complete system?	Integration prototyping and interface specification

The Spiral model does not have fixed phases —

each loop is tailored to what the project needs most at that point in time. This flexibility makes it powerful but harder to manage.

Spiral Model — Strengths, Limitations and When to Use

Strengths

- Risk management is built into every cycle — not an afterthought
- Combines structure with flexibility — Waterfall within each loop
- Prototyping used strategically, at every loop as needed
- Customer reviews built in at the end of every loop

Limitations

- Requires expertise in risk assessment — not all teams have this skill
- Can be difficult to manage — no fixed phases means more complexity
- Hard to apply to contracts — customers want a fixed price and scope
- Overkill for small projects where risks are well understood

Use Spiral when:

Large and complex systems · High technical or requirements risk ·
Long-duration projects · Safety-critical or security-critical systems

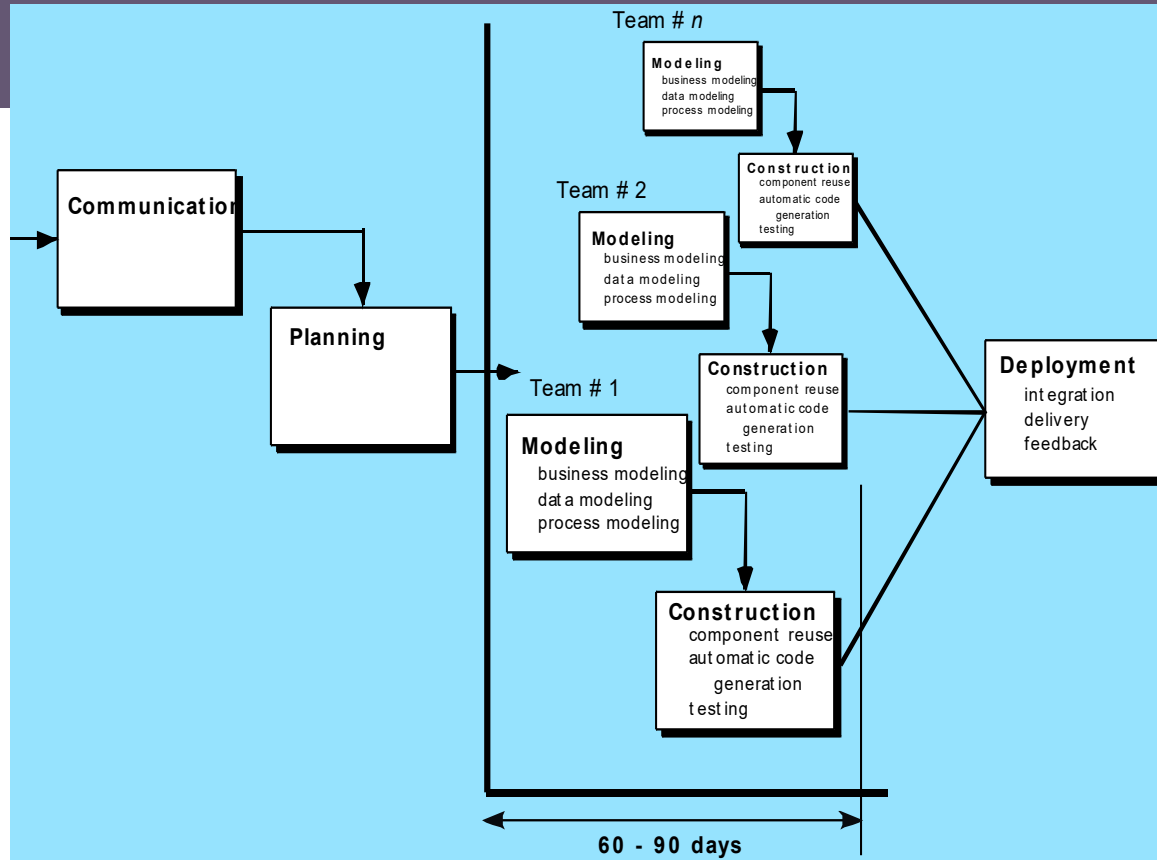
Spiral model influenced the creation of Risk-driven development practices, and its ideas appear in modern DevSecOps approaches where security risks are addressed continuously.

Plan-Driven Model · 2.3.6

Rapid Application Development (RAD)

Extremely short cycles · Parallel teams · Component reuse · Time-boxed delivery

The RAD Model



RAD Model — Overview and Concept

An incremental model that prioritises extremely short development cycles

Rapid Application Development (RAD) is an incremental software development process model that emphasises an extremely short development cycle. If requirements are well understood and project scope is constrained, RAD enables a development team to create a fully functional system within 60 to 90 days.

RAD uses multiple parallel teams, each working on one modular piece of the system simultaneously:

Team 1	Team 2	Team 3
Business modeling	Business modeling	Business modeling
Data modeling	Data modeling	Data modeling
Process modeling	Process modeling	Process modeling
App. generation	App. generation	App. generation
Testing & turnover	Testing & turnover	Testing & turnover

60–90 day timebox · All teams work in parallel · Results integrated at the end

RAD Model — Phases and Problems

1	Business modeling	Model the information flow through business processes. What information drives business processes? Who generates it? Where does it go? What decisions use it?
2	Data modeling	Refine the information from business modeling into data objects needed to support the business. Identify the characteristics (attributes) of each object and the relationships between them.
3	Process modeling	Transform the data objects defined in data modeling into the information flows needed to implement a business function. Processing descriptions are created for adding, modifying, deleting, or retrieving a data object.
4	Application generation	RAD assumes use of 4th generation languages (4GL) or visual component-based tools to automate system construction from reusable components. Code generation is used wherever possible.
5	Testing and turnover	Since RAD reuses existing components, many of them are already tested. New components and interfaces must be fully tested.

RAD problems: Requires many developers · Customer must be committed to rapid-fire activities · Cannot be modularised easily → RAD fails · High technical risk systems are not suited

Reuse-Oriented Software Engineering

Assemble systems from existing components · Reduce cost and risk · Adjust requirements to fit





Reuse-Oriented Engineering — Overview

Also called Component-Based Software Engineering (CBSE) or Integration and Configuration

Rather than developing software from scratch, the system is assembled from existing reusable components, frameworks, libraries, and off-the-shelf (COTS — Commercial Off-The-Shelf) software systems. Requirements are adapted to fit available components where necessary.

Three types of reusable software that form the basis of this approach:

Web services

Services developed to standards (REST, SOAP) that are available for remote invocation over the internet. Example: payment gateways, mapping services, authentication providers.

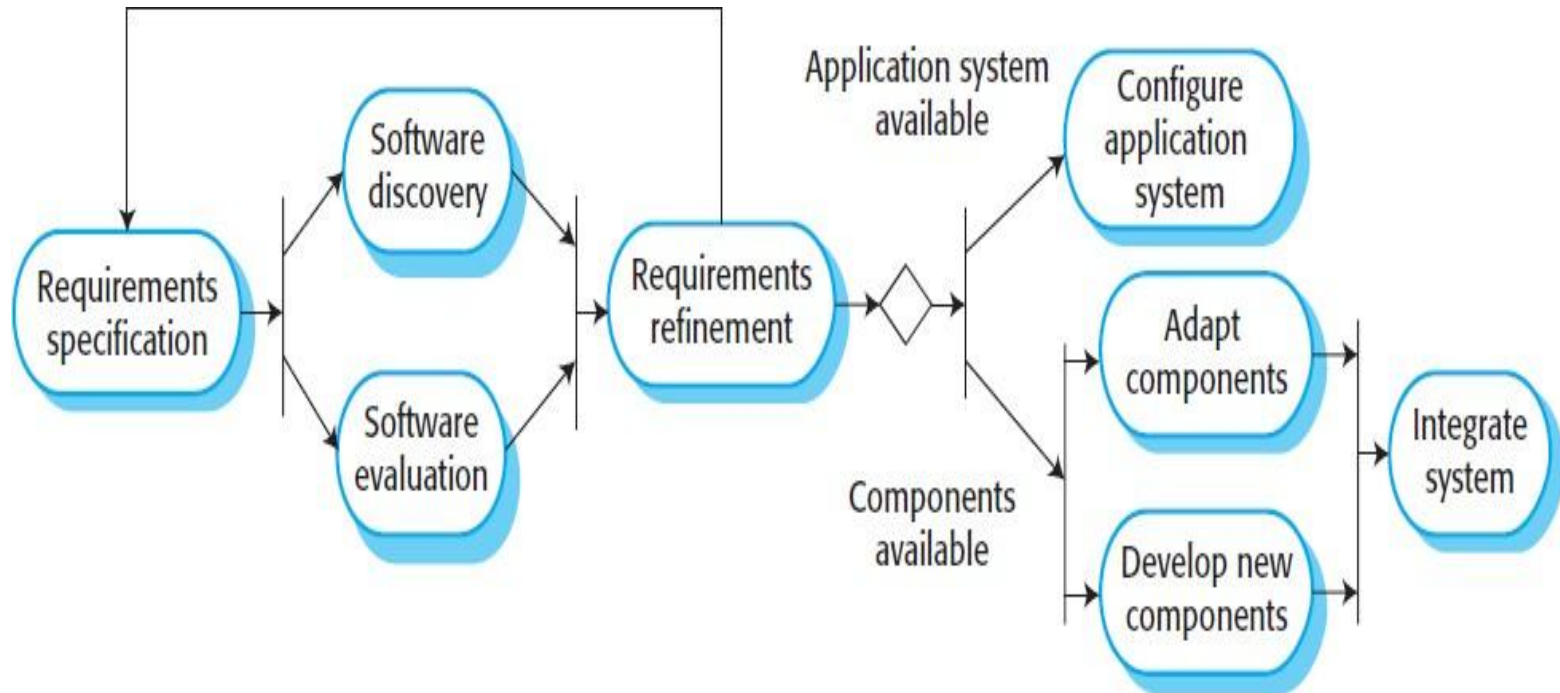
Collections of objects

Component packages — frameworks like Spring, Django, React — developed as a package to be integrated into an application framework. Used in most modern development.

COTS systems

Stand-alone application systems configured for use in a particular environment. Example: Microsoft Dynamics (ERP), Salesforce (CRM), Moodle (LMS). Configured, not customised.

Process Stages of Reuse-Based Development Component-based Development



Reuse-Oriented Engineering — Process Stages

Unlike Waterfall, requirements are not fixed before development begins — they are refined based on what reusable components are available. This is a critical difference from all other plan-driven models.

1	Requirements specification	Initial requirements for the system are specified — not in full detail, but sufficiently to guide the component search. Brief descriptions of essential requirements and desirable features.
2	Software discovery and evaluation	A search is made for components and systems that provide the required functionality. Candidate components are evaluated against the essential requirements and general suitability.
3	Requirements refinement	Requirements are refined using information about the reusable components discovered. If modifications to requirements are impossible, the search is repeated to find alternatives.
4	Application system configuration	If an off-the-shelf application system that meets requirements is available, it is configured for use to create the new system. This is the ideal outcome.
5	Component adaptation and integration	If no off-the-shelf system is fully adequate, individual reusable components are modified (adapted) and new components are developed. These are then integrated to create the system.

Principle: if the available component almost meets your requirement, consider changing the requirement — not the component.

Reuse-Oriented Engineering — Benefits and Risks

Benefits

- Significantly reduces the amount of software that must be developed from scratch
- Reduces development cost — you are not paying to write code someone else already wrote
- Reduces risk — reusable components are already tested and proven in production
- Faster delivery — less code to write means shorter development time
- Higher quality — components from reputable sources are often better than bespoke code

Risks and Limitations

- Requirements compromises — you must accept what the component does, not what you want
- Loss of control — if a third-party component changes, your system may break
- Security risks — vulnerabilities in third-party libraries affect your system (Log4Shell, etc.)
- Vendor lock-in — dependency on a commercial component that may be discontinued
- Integration complexity — not all components work well together

Reuse-oriented engineering is not a choice anymore — it is the default. Every modern software system uses frameworks, libraries, APIs, and cloud services. The discipline is in choosing, evaluating, and managing those dependencies well.

Plan-Driven Models — Comparison Summary

Use this table to compare models at a glance. The right model depends on project context, not preference.

Model	Key idea	Requirements?	Best when	Main risk
Waterfall	Sequential phases	Must be stable	Small, stable, regulated	Late defect discovery
Incremental	Build in pieces	Priority-ordered	Large systems, fast value	Architecture degradation
V&V	Check it's right	Formal spec needed	All projects need this	Testing only at end
Prototyping	Build to learn	Vague/unkn own	Unclear requirements	Prototype becomes product
Spiral	Risk-driven loops	Can evolve	Large, high-risk systems	Needs risk expertise
RAD	Parallel fast teams	Well understood	Short-deadline projects	Modularity required
Reuse-Oriented	Assemble components	Adjusted to fit	Rich component ecosystem	Requirements compromise

The Four Core Process Activities

Every software process — regardless of model — includes these four activities

While different models organise work differently, all software processes include the same four fundamental activities. Understanding these activities helps you understand what any process model is asking you to do.

1 Software Specification

Defining what services the system should provide and the constraints on its operation and development.

2 Software Design & Implementation

Defining the organisation of the system and translating that organisation into executable code.

3 Software Validation (V&V)

Showing that the system conforms to its specification and meets the requirements of the customer.

4 Software Evolution (Maintenance)

Changing the system in response to changing customer needs, corrected defects, and new technology.

These four activities are present in every model — but their sequence, overlap, and depth varies. Waterfall sequences them strictly. Agile interleaves them in each sprint.

Coping with Change

Change is inevitable — the question is how your process handles it

Change leads to rework. Rework costs time and money. A good process minimises the cost of accommodating change — it does not pretend change will not happen.

Change anticipation

Build activities into the process that predict changes before significant rework is required.

Example: Develop a prototype early to expose risks and get customer feedback before full development begins.

Used in: Spiral model, prototyping-first approaches.

Change tolerance

Design the process and software so changes can be easily incorporated.

Example: Incremental development — proposed changes go into a future increment rather than disrupting the current one.

Used in: Agile methods, incremental delivery.

Continuous refactoring

Continuously improve internal code structure without changing external behaviour.

This prevents gradual architectural degradation as changes accumulate over time.

Used in: Agile, XP, DevOps workflows.

2.4

The Unified Process

A hybrid model bridging plan-driven and agile cultures

The Unified Process (UP)

"Use-case Driven, Architecture-Centric, Iterative and Incremental"

The Unified Process is a hybrid — it combines Waterfall's structured phases and milestone reviews (plan-driven culture) with iterative, incremental development within each phase (agile culture). It is the process that deliberately bridges both worlds.

Use-case Driven

Requirements captured as use cases — descriptions of how users interact with the system. Each iteration implements a set of use cases end-to-end.

Architecture-Centric

A validated architectural baseline is the most important early deliverable. Architecture shapes the whole system — it must be proven before full construction begins.

Iterative & Incremental

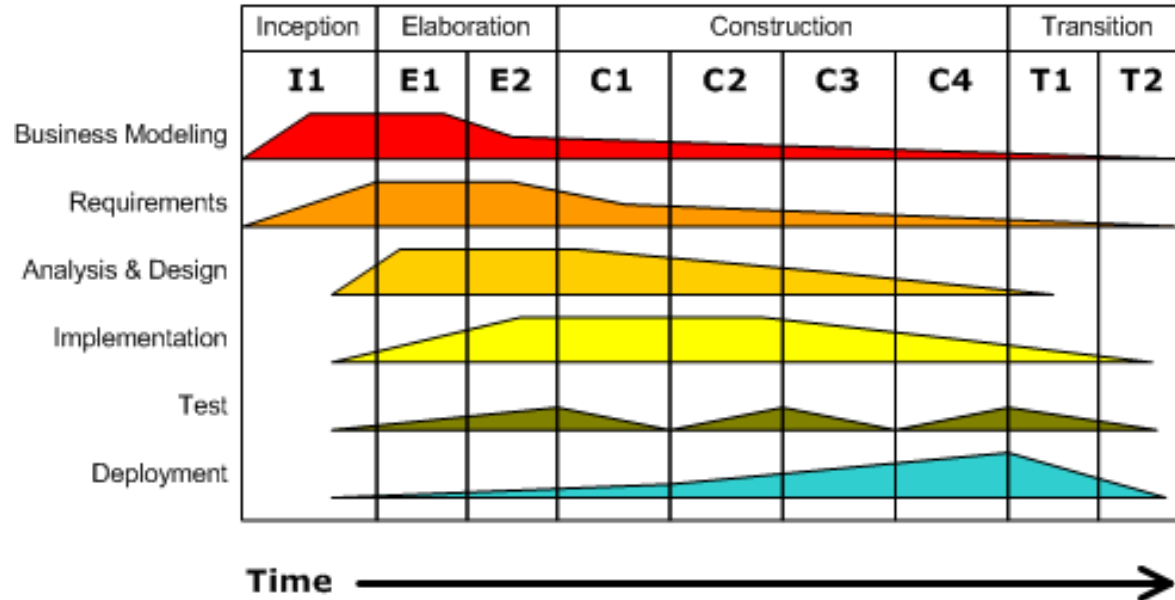
Four phases each containing multiple time-boxed iterations. All disciplines (requirements, design, test) run in parallel with varying intensity.

Risk-Focused

Most critical risks tackled early — especially in Elaboration. The phase structure ensures risks are addressed before full construction investment.

Iterative Development

Business value is delivered incrementally in time-boxed cross-discipline iterations.



This diagram illustrates the relative emphasis of different disciplines wrt to phases and time

UP Work Products

Inception phase

Vision document
Initial use-case model
Initial project glossary
Initial business case
Initial risk assessment.
Project plan,
phases and iterations.
Business model,
if necessary.
One or more prototypes

Elaboration phase

Use-case model
Supplementary requirements
including non-functional
Analysis model
Software architecture
Description.
Executable architectural
prototype.
Preliminary design model
Revised risk list
Project plan including
iteration plan
adapted workflows
milestones
technical work products
Preliminary user manual

Construction phase

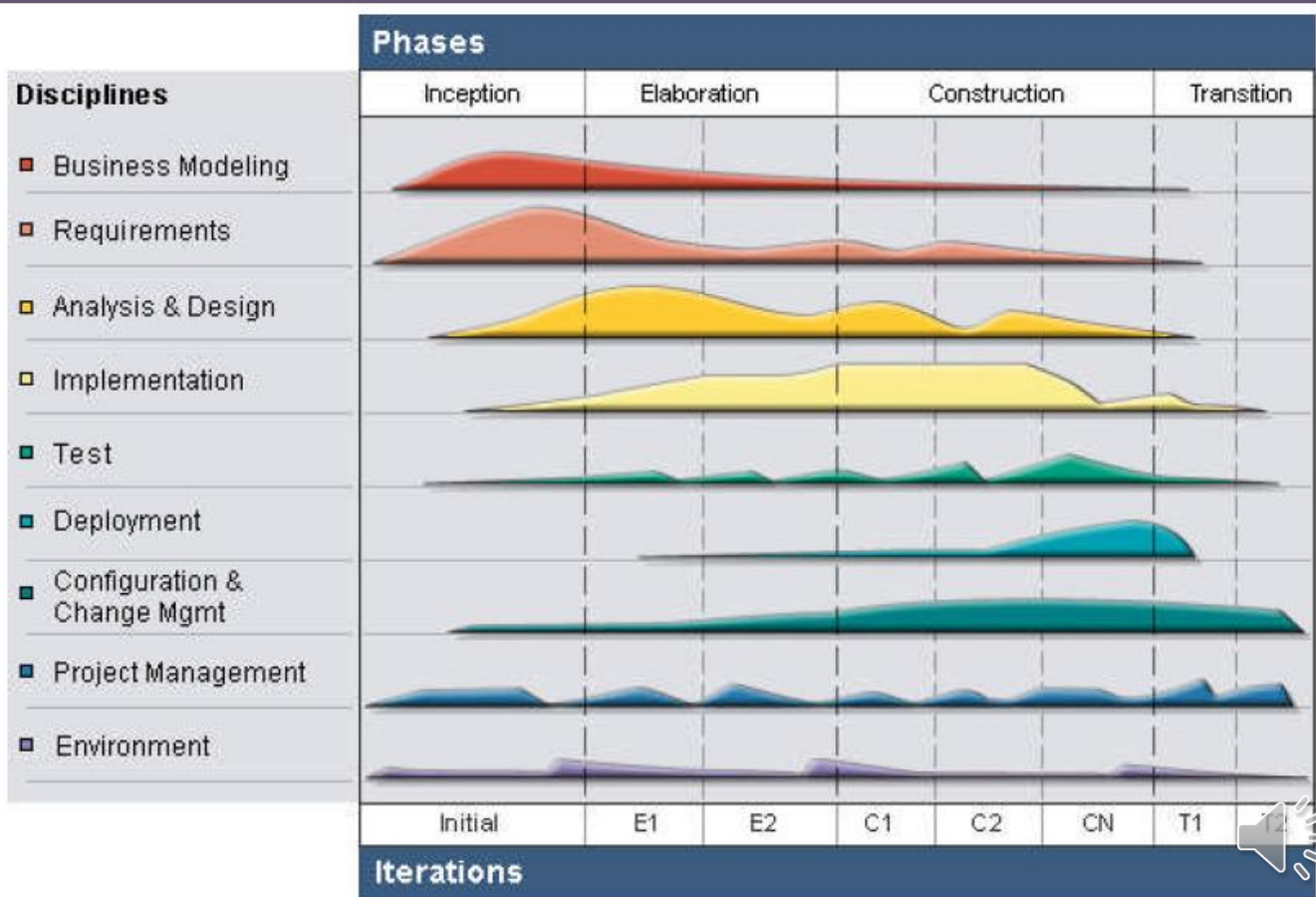
Design model
Software components
Integrated software
increment
Test plan and procedure
Test cases
Support documentation
user manuals
installation manuals
description of current
increment

Transition phase

Delivered software increment
Beta test reports
General user feedback



RUP: Rational Unified Process



UP — Four Phases and Disciplines

Phases represent business milestones · Disciplines run in parallel throughout

Inception Phase 1	Elaboration Phase 2	Construction Phase 3	Transition Phase 4
Establish business case and project scope	Capture requirements, establish and validate architecture	Build the full system incrementally	Deploy to real users, collect feedback
• Initial use cases • Risk identification • Preliminary plan • Go/no-go decision	• Architecture baseline • Address major risks • Detailed plan • Prototype	• Implement use cases • Unit & integration test • Documentation • Multiple iterations	• Beta testing • User training • Bug fixes • Final release

Disciplines (run throughout all phases — with varying effort):



Key insight in UP: testing begins in inception — not after coding. Requirements are refined throughout — not just at the start.

2.5

Agile Culture — Introduction

The cultural shift that reshaped software development

The Agile Manifesto (2001)

Four values that defined a new approach to software development

In 2001, 17 experienced practitioners met in Snowbird, Utah — frustrated with heavyweight plan-driven processes. They agreed on four shared values that became the Agile Manifesto.

Individuals and interactions

over

processes and tools

Working software

over

comprehensive documentation

Customer collaboration

over

contract negotiation

Responding to change

over

following a plan

*"That is, while there is value in the items on the right, we value the items on the left more." —
The Agile Manifesto (agilemanifesto.org)*

Agile in Practice — Scrum at a Glance

The most widely used agile framework — covered in full depth in Section 3

Scrum provides just enough structure for a self-organising team to deliver working software in short, repeatable cycles called sprints (1–4 weeks). It is not a methodology — it is a lightweight framework.

Three roles

Product Owner

Owns the backlog — prioritises what gets built and why. Represents customer interests.

Scrum Master

Coaches the team, removes impediments, protects the sprint from interruption.

Dev Team

Cross-functional, self-organising team of 3–9 people. No sub-teams or titles.

Sprint cycle (1–4 weeks)

Sprint Planning

Select backlog items

Daily Stand-up

15 min — what done, what next, blockers

Sprint Execution

Build, test, integrate

Sprint Review

Demo to stakeholders

Retrospective

Improve the team process

Next section covers Agile methods in full depth — Scrum, XP, Kanban, Lean, and how to choose between them.

2.6

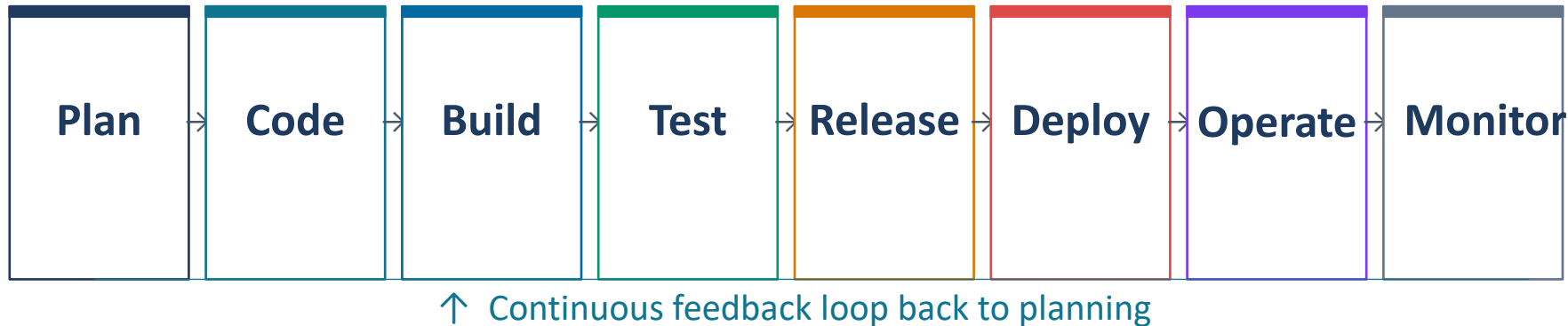
DevOps and AI in Software Processes

The modern development and delivery culture

DevOps — Development and Operations Together

The culture that removed the wall between building and running software

Traditionally, Development (builds software) and Operations (deploys and runs it) were separate teams — often in conflict. DevOps is the culture and set of practices that unifies them around shared responsibility for the full lifecycle.



Continuous Integration (CI)

Code merged and tested automatically, many times per day. Problems caught early, not at release.

Continuous Delivery (CD)

Software always in a deployable state. Releases are routine, low-risk events — not stressful milestones.

Infrastructure as Code

Environments configured through code (Docker, Terraform) — deployments are repeatable and auditable.

AI is Changing Software Processes

A forward look — preparing you for the profession you are entering

AI tools are being integrated into every stage of the software development process. This does not replace software engineering — it changes the shape of the work engineers do.

Requirements

AI extracts and structures requirements from meeting transcripts and user interviews. NLP identifies conflicts.

Design

AI generates candidate architectures, suggests patterns, and flags scalability or security issues early.

Coding

GitHub Copilot, ChatGPT generate boilerplate, complete functions, explain unfamiliar code in the IDE.

Testing

AI generates test cases from specs, identifies uncovered code paths, detects anomalies automatically.

Operations

AIOps platforms analyse logs, predict failures, auto-scale, and route incidents without human triage.

What AI cannot do: understand context, exercise professional judgement, take responsibility, design trustworthy systems, or resolve ethical dilemmas. That remains the software engineer's domain.

Choosing the Right Process Model

There is no single right answer — context determines the best fit

Factor	Plan-Driven suits when...	Agile suits when...
Requirements stability	Clear, stable, unlikely to change	Uncertain, evolving, customer-driven
System criticality	Safety-critical (aviation, medical, defence)	Commercial product (app, website, SaaS)
Team size / location	Large, distributed, contractual	Small, co-located, collaborative
Customer availability	Available mainly at milestone reviews	Available for frequent feedback
Delivery timeline	Long-term fixed contract	Fast delivery needed, scope flexible
Technology maturity	Well-understood domain and stack	New domain, exploratory, experimental
Risk level	Risks well understood and low	High risk — use Spiral

Unified Process: large complex systems needing both structure and flexibility — the deliberate hybrid choice.

Section 2 — What We Have Covered

Check your understanding against the learning outcomes

LO1

A software process defines activities, products, roles, and conditions required to develop a software system. A defined process brings consistency, measurability, and continuous improvement.

LO2

Plan-driven culture: plan upfront, control change. Agile culture: embrace change, deliver incrementally. All models belong to one or blend both.

LO3

Waterfall — 5 sequential phases, each signed off before the next. Best for stable requirements. Limitations: late testing, inflexible to change.

LO4

Incremental, V&V, Prototyping, Spiral, RAD, Reuse-Oriented — each model solves a specific problem. The comparison table helps you choose.

LO5

The Unified Process is use-case driven, architecture-centric, iterative, and risk-focused — four phases with parallel disciplines. The deliberate hybrid.

LO6

The four core activities — Specification, Design & Implementation, Validation, Evolution — appear in every process model.

LO7

DevOps unifies Dev and Ops through CI/CD, automation, and shared responsibility. It is the modern delivery culture.

LO8

AI is being integrated into every SDLC phase — from requirements to operations. Engineering judgement and responsibility remain irreplaceable.

Coming next — Agile Software Development in depth (Scrum · XP · Kanban · Scaling Agile · Choosing your method)

References & Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th ed., Addison-Wesley, 2015

Ch.2 (Software Processes), Ch.3 (Agile) — required for Section 2

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th ed., McGraw-Hill, 2020

Ch.2–4: process models, Agile, Unified Process

Online Resources

- agilemanifesto.org — the original Agile Manifesto with all 12 principles
- scrumguides.org — free official Scrum Guide (Schwaber & Sutherland, 2020)
- Boehm, B. (1988) 'A Spiral Model of Software Development and Enhancement' — IEEE Computer
- DevOps Handbook (Kim, Humble, Debois, Willis) — recommended background reading

Coming Up Next

Section 3:

Agile Software Development

Scrum in depth · Extreme Programming · Kanban · Scaling Agile · Choosing the right approach

Tutorial this week:

SCS1303 · Prof. K. P. Hewagamage

Given a scenario, select the most appropriate process model and justify your choice · Compare any two plan-driven models